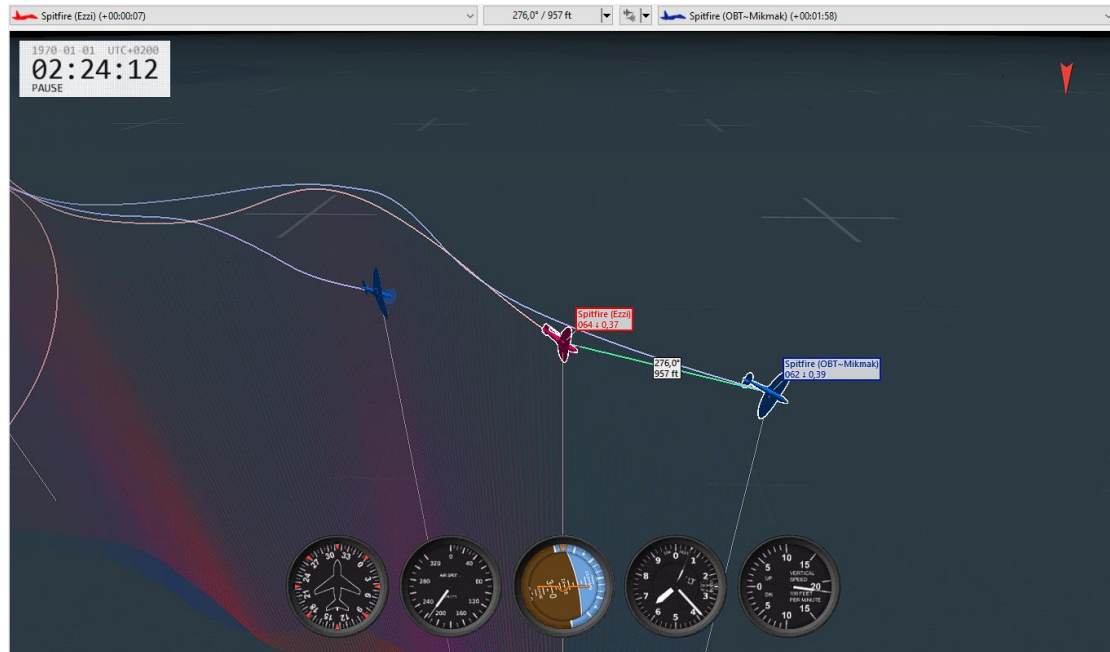


Tacview Recorder V2.0

for IL2-Sturmovik: Cliffs of Dover Blitz & Tobruk

By FlyBy (formerly OBT~Eazy).



World's first debriefing on Tacview using COD flight datas (OBT training flight)

<https://obteam.forumsactifs.net>

V2.0 features

- **No more code to integrate into your mission.**
- [New base class](#) for mission.
- Land, naval and static ([Actor](#)) units (other than players and aircraft) are displayed in [Tacview](#).
- New method for matching 3D models from CloD and Tacview.
- Damage, death or destruction of an actor managed and displayed in Tacview.
- Color change for damaged or destroyed actors.
- 3 modes available (dogfight, normal, big mission)
- Events involving players injected into the Tacview event log.
- Displaying bomb explosions in Tacview.
- On-demand creation and deletion of landmarks.
- Reduce size of the Tacview file (zip compression).
- Runs in its own [thread](#).
- Improved timers (a single mission can be accelerated up to 16x).
- Ready for future DLC ([3D model matching](#)).
- Flight data compatible with Tacview format (acmi 2.1).
- Low impact on performance.
- Programmable start, pause, stop, delayed start of data recording.
- The data for each player and the AIs are recorded in a single file.

Limitations

- As with the V1, the ballistics of projectiles (bullets, shells, bombs) are not available.
- Flight data files are stored on the server running the mission.

Foreword.

As the developer of this add-on, I am neither directly nor indirectly involved in the development, maintenance, or technical support of the Tacview software, and I am not part of Raia Software in any form whatsoever.

My personal opinion is that the Tacview software is very useful for debriefing our flights, as well as offering good value for money. In addition, the file format used by Tacview is public, free and has a simple structure.

These were the reasons why I decided to use my spare time to develop this add-on. I hope it will meet all your expectations.

This new version was an opportunity to recreate the [Tacview Recorder](#) from scratch (no backwards compatibility with the previous version).

The main aim of the rewrite was to enable the [Recorder](#) to be used with as little code as possible to be integrated by the mission designer. To achieve this, a new mission class, [TacviewMission](#), was developed. This class is derived from the standard `AMission` class. By deriving its missions from the new class, the designer no longer must add any lines of code to activate the [recorder](#), as the necessary events are intercepted and processed automatically by the new class.

We also needed to ensure that land-based [actors](#) other than aircraft, such as ships, trains, tanks, flak, etc., whether mobile or not, were processed and visible in Tacview, as well as damage and bomb explosions.

The recorder could be used for any type of mission, whether it was a single mission, a private server (lounge) or a dedicated server.

Particular attention has been paid to performance. The recorder runs in its own [thread](#) (execution flow), and the time taken to intercept and process events has been reduced to a minimum (see the "[How it works](#)" chapter).

Disclaimer of liability.

This add-on is provided as is, without any express or implied warranty as to its quality, performance, or fitness for a particular purpose. The user assumes all risks associated with its use. In no event shall the author of the add-on or any person associated with its development be liable for any direct, indirect, incidental, special or consequential damages arising out of the use of or inability to use this software, even if advised of the possibility of such damages. The user is responsible for the use of the add-on and must comply with all applicable laws and regulations. Users are advised to back up their data regularly to avoid any loss in the event of software malfunction. Use of this add-on implies unreserved acceptance of the terms of this disclaimer.

Let us pay tribute with respect and gratitude to all the pilots, bombers, gunners, mechanics and every man and woman who, with unwavering courage and dedication, offered the ultimate sacrifice for their homeland, bravely defending the skies of their nation, whatever side they were on.

Acknowledgements

- Thanks to OBT~Mikmak, OBT~Lionel and OBT~Brunov for their participation in the multiplayer tests.
- Thanks to ATAG_Lenny for testing the recorder in single mission mode.
- Thanks to Artist and Scott for their help in trying to correct the geographical coordinates.
- Thanks to the past, present and future volunteers on the Team Fusion Simulations team for maintaining and improving this simulator in their spare time.
- Thanks to the designers of the single-player and multi-player missions, whose hard work and rigor help to keep players interested.
- And, of course, thanks to the players behind the scenes or the ace of aces who, through their fair play and good humor, keep me flying.

Table of contents.

Lexicon.	6
How it works.	7
Integration into your missions.	9
New mission.	9
Existing mission (the mission already contains scripts).	9
Basic methods used by the recorder	11
Setting up and controlling the recorder.	12
Properties.	12
DestinationFolder	12
MissionType	12
ShowPlayer	12
ShowPlayerAsHuman	13
StartDelay	13
ZipFinalFile	13
Methods.	14
StartRecorder()	14
PauseRecorder()	14
StopRecorder()	14
DisableRecorder()	14
AddWaypoint(name, x, y, z, army= 0)	14
RemoveWaypoint(name)	14
AddBookmark(message)	15
Representation of damage.	16
Events in Tacview.	18
Saved events.	18
Display the event log.	18
Select an event.	19
Matching 3D models.	20
Table of mappings (aircraft only).	20
Useful links.	22
Known issues.	23

Lexicon.

Actor : any object, moving or not, that can interfere with the game (player, plane, crew, ship, tank, train, flak, etc.).

CSV file (Comma-Separated Values): an organized list of data where each row represents a record and each column represents an attribute or characteristic. It is a simple format commonly used to store tabular data, often in spreadsheets or databases.

Multithreading : multithreading is widely used in computer applications for tasks such as processing data in the background, manipulating user interfaces, executing several operations simultaneously, and many others. It improves the responsiveness of applications, optimizes the use of system resources and speeds up the processing of parallel tasks.

Recorder: shortcut for Tacview Recorder

Tacview: flight data visualization and analysis software developed by RAIA Software Inc. Available at <https://www.tacview.net>

Tacview Recorder : add-on developed to intercept and record flight data, mission objects, damage and explosions compatible with the Tacview software format (generates a file in acmi format).

TacviewMission: new base class for Tacview Recorder. This class intercepts the events required by the recorder. It is derived from the AMission class.

Buffer : a buffer is a temporary area where the computer stores data waiting to be used. It is a transit zone where data passes through before being processed or moved to its destination.

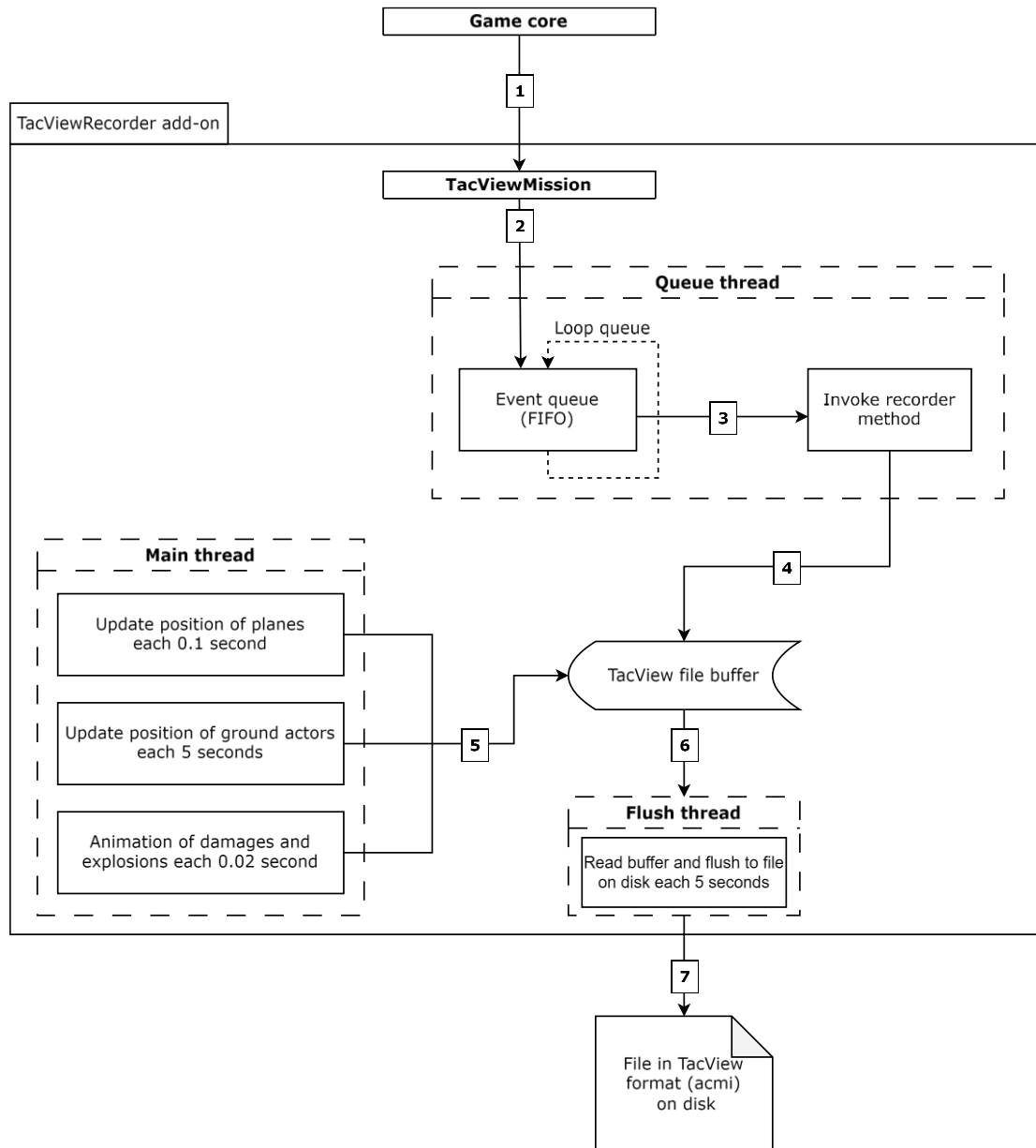
Thread (execution flow): in IT terms, a thread is a unit of execution within a running program, and a thread is a part of this program that can be executed independently of the other parts. The recorder uses its own threads to process events triggered during the game (actor creation, damage to an actor, death, destruction, explosion, etc.) and does not interfere with the main game thread. This guarantees minimal performance degradation.

Watchdog : developed by Colander and maintained by Artist from the TFS team. This program manages the stopping and starting of missions on a dedicated server (not a lounge).

How it works.

This chapter briefly presents the operation of the [recorder](#) in technical terms and may lead to some confusion on the part of readers who are not familiar with programming techniques and the operation of a computer software.

Simplified representation of the recorder architecture.



The recorder is made up of 3 independent [threads](#) and a [buffer](#) memory (which limits writing operations to the computer's hard disk). This architecture makes it possible to separate the execution of the recorder code from that of the game, and therefore to affect the performance of our favorite simulator as little as possible, whether on a single mission or in multi-player

The "queue" thread intercepts events from the game. The term event is not used correctly here because technically it's a function call, but for the most part these calls are linked to game events such as damage, destruction, player connection, etc., and for the sake of clarity that's the term we'll use.

The "main" thread is responsible for updating the positions of the [players](#) (players, aircraft, ground units) as well as the damage and explosion animations.

To limit the size of the Tacview

file on disk, the timescales for the various updates are optimized according to the type of player. For players and aircraft, updates are executed every 1/10th of a second, while for ground units, which are slower, 5 seconds are sufficient. Damage and explosion animations, if any, are processed every 2/100ths of a second.

The "flush" thread writes the contents of the buffer to the file (on the hard disk) in Tacview format every 5 seconds.

The file name is formatted as follows:

`Tacview-YearMonth-Day-HourMinuteSecond-COD-MissionName.txt.acmi`

If the file is compressed, the extension is zip.acmi

The main thread uses the time taken from `Time.current()`, which corresponds to the real time of the mission, so the recorder can run even if the mission is accelerated, decelerated or paused

Threads are synchronized using a flag to prevent deadlock, which is not shown here.

Note: the order of the operations described below is not representative of the actual order in which they are performed, as the recorder operates in a [multithreading](#) environment.

- 1** The game core sends an event to the TacviewMission class
- 2** This event is intercepted by the recorder, which places it on a stack and returns the hand to the game core immediately (the game does not have to wait for this event to be processed and its performance is not affected).
This stack is organized in such a way that the 1^{er} element entered is the 1^{er} element exited (FIFO) by the loop running through the stack. These operations take place in the "events" thread.
- 3** The internal method corresponding to the event is executed.
- 4** The method may send data to the buffer depending on the event (creation of a player or actor, messages relating to the event, etc.).
- 5** The "main" thread sends positions (aircraft, player and ground units) and "animations" to the buffer.
- 6** and **7** The "flush" thread retrieves the contents of the buffer and writes them to the file on disk.

Integration into your missions.

This chapter does not cover the creation of missions and scripts. We assume that you have sufficient knowledge to understand what follows. If not, you will certainly find help on the [ATAG](#) forum and in particular in the section [Full Mission Builder and Scripts](#).

This new version enables you to activate the recorder without having to add any code to your missions, thanks to the new TacviewMission base class.

New mission.

If a .cs file does not yet exist for your job, you can use the template (Mission_Template.cs) included in the installed files. This template is available in the following folder
C:\Users\UserName\Documents\1C SoftClub\1-2 sturmovik cliffs of dover\TacviewTemplate
Copy the MissionTemplate.cs file to the location of your mission (where your .mis file is) and rename it with the name of your mission.

For example, if your mission is called Dogfight.mis, rename the template to Dogfight.cs

Make sure the extension is **.cs**

All you must do is launch your mission, and the recorder will record the data automatically.

Existing mission (the mission already contains scripts).

- Add the reference to TacViewRecorder.dll at the beginning of the mission .cs file.

```
// $reference TacviewRecorder.dll
using TacviewRecorder;
```

Please note: do not delete references that already exist in your file.

- Replace the **AMission** base class with **TacviewMission**.

```
public class Mission : AMission
```

becomes

```
public class Mission : TacviewMission
```

Warning: if you call one of the methods used by the TacviewMission base class (see [methods used](#)), make sure you call the base method in your code.

For example, if you intercept the OnPlaceEnter :

```
public override void OnPlaceEnter(Player player, AiActor actor, int placeIndex)
{
    base.OnPlaceEnter(player, actor, placeIndex); // line to add

    // YOUR CODE HERE //
}
```

NB: it is good programming practice to always call the base class method in the case of inheritance.

Note: any secondary missions loaded by the `gpPostMissionLoad` method do not have to inherit from TacviewMission class but they can. The scripts linked to these missions have no impact on the recorder.

In addition to the comments concerning [the methods used](#) by the recorder, you should also consider the possible use of the excellent scripts available on the ATAG forum.

<https://theairtacticalassaultgroup.com/forum/showthread.php?t=34571>

You must ensure that these scripts call the functions of the TacviewMission base class.

Before stopping the scripted mission by calling the

`Process.GetCurrentProcess().Kill()` method, be sure to call the

`Battle.OnBattleStoped()` method in order to free up the resources used by the recorder and close the Tacview file correctly to avoid this file becoming corrupted.

The same applies if you are using the [watchdog](#) (mission rotation).

<https://theairtacticalassaultgroup.com/forum/showthread.php?t=35224&highlight=watchdog>

https://www.team-fusion-simulations.de/tfsresources/?&path=03_Servers

Basic methods used by the recorder.

If your mission scripts (.cs file) call one of the following functions, it is essential to call the base class method in your code to ensure that the recorder works properly, see the ["Integrating the recorder"](#) chapter.

For example, if you intercept the OnPlaceEnter :

```
public override void OnPlaceEnter(Player player, AiActor actor, int placeIndex)
{
    base.OnPlaceEnter(player, actor, placeIndex); // line to add

    // YOUR CODE HERE BELOW //
}
```

NB: it is good programming practice to always call the base class method in the case of inheritance.

```
Init
OnActorCreated
OnActorDamaged
OnActorDead
OnActorDestroyed
OnAircraftCrashLanded
OnAircraftCutLimb
OnAircraftDamaged
OnAircraftKilled
OnAircraftLanded
OnAircraftLimbDamaged
OnAircraftTookOff
OnBattleInit
OnBattleStarted
OnBattleStoped
OnBombExplosion
OnBuildingKilled
OnMissionLoaded
OnPersonHealth
OnPersonMoved
OnPersonParachuteFailed
OnPersonParachuteLanded
OnPlaceEnter
OnPlaceLeave
OnPlayerArmy
OnPlayerConnected
OnPlayerDisconnected
OnStationaryKilled
```

Configuration and control of the recorder.

Properties.

These properties **MUST** be used in your mission's *public override void Initd()* method, otherwise they are ignored.

DestinationFolder

Choose the destination folder for the Tacview file.

Type: character string

By default, the destination folder is:

C:\Users\NameDocuments\1C SoftClub\il-2 sturmovik cliffs of dover\Tacview

Example:

```
DestinationFolder = @"D:\Tacview\";
```

MissionType

Defines the type of mission in progress. Depending on the type, certain objects are ignored.

Type : Enum TypeOfMission

Possible values :

TypeOfMission.BigMission = Aircraft + ground units + static objects.

TypeOfMission.Dogfight = Aircraft only.

TypeOfMission.Normal = Aircraft + ground units.

The default mission type is BigMission

ShowPlayer

In Tacview, displays players in the form of a green pyramid.

Type: Boolean (true/false)

By default, players are displayed in Tacview (value = true).

Example :

```
ShowPlayer = true;
```



(ShowPlayer = true)

If several players are on board the plane, they are stacked.



ShowPlayerAsHuman

In Tacview, displays the player as a green character.

Type: Boolean (**true/false**)

By default, the players are represented in the form of a green pyramid.

To be used with the `ShowPlayer` property.

Example:

```
ShowPlayerAsHuman = true;
```



ShowPlayerAsHuman = true (ShowHuman must be true)

StartDelay

Sets a delay (in seconds) before the recorder starts up.

Type: Integer (int)

Useful if the mission designer considers that there is no action to be recorded in Tacview before a certain time.

Possible value

Value = 0 : recorder starts immediately

Value > 0: starts x second(s) after the mission is loaded.

Value = -1: the recorder does not start automatically and must be started using the `StartRecorder()` function.

The timer can be cancelled by calling the `StartRecorder()` function, which cancels the timed start and starts the recorder immediately.

The `StopRecorder()`

method cancels the timed start and permanently stops the recorder.

The `PauseRecorder()`

method has no effect on the timed start.

By default, the recorder starts automatically when the mission is loaded.

Example:

```
StartDelay = 300; // Start in 300 seconds
```

ZipFinalFile

The Tacview file is compressed in ZIP format. Reduces the size of the uncompressed file by up to 70%.

Type: Boolean (true/false)

By default, the Tacview file is compressed.

Example:

```
ZipFinalFile = false; // do not compress the file
```

Methods.

StartRecorder()

Starts the recorder on demand after pausing or delaying start-up. See the `PauseRecorder()` and `StartDelay()` methods.
Can be called by a trigger or a menu entry.
By default, the recorder starts automatically when the mission is loaded.

PauseRecorder()

Suspends the recorder.
The recording of : positions, damages, explosions and events is suspended.
Damages and destructions are, however, kept in memory and processed when the recorder is restarted (change status and color of actors).
Can be called by a trigger or a menu entry.

StopRecorder()

Permanently stops the recorder. **Cannot be restarted. Not to be confused with the `PauseRecorder()` command.**
Can be called up by a trigger or a menu entry.

DisableRecorder()

Prevents the recorder from starting up. Allows you to run a mission without taking the recorder into account and without having to change the mission's base class.
All calls to recorder methods are ignored.
To be called in the method: `public override void Init()` method of the base mission.

AddWaypoint(name, x, y, z, army= 0)

Positions a reference point in Tacview at X, Y and Z coordinates, identified by a unique name.
The coordinate values must be equal to or greater than 0.
The army parameter (0 by default) defines the color of the reference point (0 = green, 1 = red, 2 = blue).



RemoveWaypoint(name)

Deletes the waypoint identified by its name.

AddBookmark(message)

Injects a message into the Tacview [event log](#).

Allows the mission designer to add a landmark to the Tacview timeline.

Warning: no effect if used in the *public override void Init()* function (the recorder has not yet started its timeline).

Preferably used in a trigger to synchronise the message correctly.

Example:

```
AddBookmark("Bombers detected off Dover.");
```

Representation of damage.

Players and [actors](#) who are injured, dead, damaged or destroyed change color in [Tacview](#).

For the players, the colors are green, yellow, white



Player OK



Injured player



Dead player
(plane turns white)

Aircraft and ground units.

Blue Army: blue, purple, white

Red Army: red, orange, white



No damage



Damaged



Inoperative OR pilot killed.

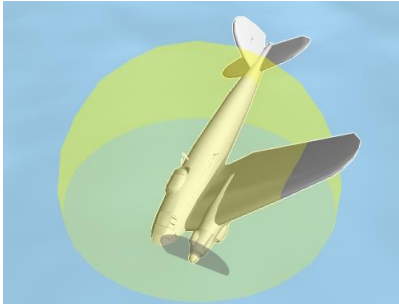
Notes:

If a player takes the place of a pilot who has been killed AND the aircraft is still operational, the aircraft takes the color defined according to its army and its condition (undamaged or damaged).

When a player or actor suffers damage, they are surrounded by a temporary yellow bubble.



A destroyed aircraft takes on a white color, surrounded by a temporary yellow bubble when it crashes.



Bomb explosions.

Bomb explosions are represented by temporary yellow bubbles (5 seconds).

The diameter of the bubble is calculated from the mass of the bomb.



If a player ejects from his plane, he is represented by a parachutist.



Parachutist orientation is currently incorrect in Tacview version 1.9.3
An anachronistic alternative is used instead.

Events in Tacview.

This chapter is not an instruction manual for the Tacview software.

Please refer to the online documentation here: https://tacview.fandom.com/wiki/The_basics

Saved events.

The recorder only injects events explicitly involving the player(s) into the Tacview event log, to make it easier to find a particular event and to avoid overloading the log.

These include the following events:

- player connects or disconnects from the server,
- player selects an army,
- player takes a seat in a plane (solo or multi-crew),
- player takes off,
- player has participated in the destruction of an actor (player, aircraft or ground unit),
- player is injured,
- player is killed by one or more players or an AI,
- player bail out, ...

Note: when several players are on board a multi-crew aircraft, each player is involved in the event (except in the event of injury) both as initiator and victim.

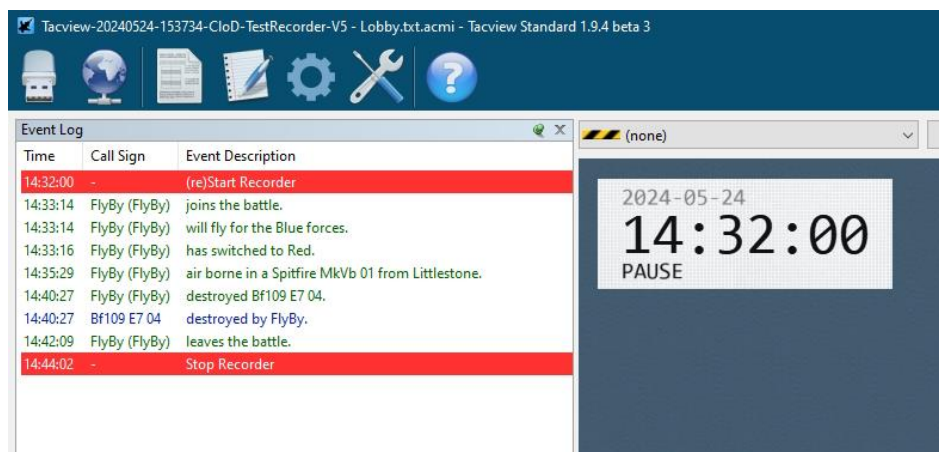
Display the event log.

- Click on the "Analysis" icon in the Tacview menu bar.
- Select "Event Log" from the list.

It is not useful to open the Tacview "flight log" as it contains automatically generated events and is not accurate enough.



If a file is loaded and contains events, they will be displayed in the log.



Select an event.

To go to the moment of the event in the Tacview timeline, click on a line. The actor initiating the event will be automatically selected by Tacview, in this case the pilot with callsign FlyBy.

The screenshot displays the Tacview Recorder V 2.0 interface. At the top is a dark blue toolbar with icons for a computer, globe, document, notepad, settings, tools, and help. Below the toolbar is the 'Event Log' window, which contains a table of recorded events. The table has three columns: 'Time', 'Call Sign', and 'Event Description'. The event at 14:35:29 is selected, highlighting the pilot 'FlyBy (FlyBy)'. To the right of the event log is a control bar with a dropdown menu showing 'FlyBy / Infantry (FlyBy)', a 'N/A' button, and a 'PAUSE' button. The main 3D view on the right shows a red Spitfire MkVb 01 aircraft in flight against a blue sky. A label 'FlyBy (FlyBy) 01' is visible next to the aircraft.

Time	Call Sign	Event Description
14:32:00	-	(re)Start Recorder
14:33:14	FlyBy (FlyBy)	joins the battle.
14:33:14	FlyBy (FlyBy)	will fly for the Blue forces.
14:33:16	FlyBy (FlyBy)	has switched to Red.
14:35:29	FlyBy (FlyBy)	air borne in a Spitfire MkVb 01 from Littlestone.
14:40:27	FlyBy (FlyBy)	destroyed Bf109 E7 04.
14:40:27	Bf109 E7 04	destroyed by FlyBy.
14:42:09	FlyBy (FlyBy)	leaves the battle.
14:44:02	-	Stop Recorder

Matching 3D models.

This chapter explains, for information purposes only, how the [recorder](#) defines the representation of actor types and models in [Tacview](#).

The 3D models are mapped between Cliffs of Dover and Tacview using a text file in [CSV](#) format generated from an Excel spreadsheet.

This file is located in the following folder:

C:\Users\UserName\Documents\1C SoftClub\1-2 sturmovik cliffs of doverTacviewParam
and is called TacviewModelMapping.csv

This method has been chosen to avoid recompiling and completely reinstalling the recorder if a match needs to be modified later (adding DLC, for example).

It is not advisable to modify the contents of the correspondence file, as this could render the recorder unusable.

When the recorder is initialized, the contents of the file are processed and organized in an internal list according to the type of [actor](#) and its internal name in the game.

When the game creates an actor, the recorder looks for a match in its list and defines the representation in Tacview.

For example, if the game creates a Spitfire IIb

aircraft with the internal name tobruk:Aircraft.SpitfireMkIIb, the aircraft in Tacview will be represented by a 3D model with the name FixedWing.Spitfire.obj and its identifier will be "Spitfire MkIIb" followed by its creation order number.

The available 3D models can be found on the [Tacview GitHub](#) site.

You will need software capable of viewing this type of file, such as Cem Yuksel's Obj Viewer, available here: <http://www.cemyuksel.com/software>
(This software is no longer updated but still works perfectly).

Some aircraft or ground units have no correspondence in Tacview. For these, we have chosen matches that are as close as possible.

For example, a Wellington, Blenheim or Beaufighter will be represented as a Mosquito, a Dornier as a B-25, or an FW 200 as a B-17.

Table of mappings (aircraft only).

Aircraft (all variants)	Tacview 3D model	Exact match
Anson	FixedWing.Mosquito.obj	N
Beaufighter	FixedWing.Mosquito.obj	N
Bf108	FixedWing.PC-21.obj	N
Bf109	FixedWing.Bf 109.obj	O
Bf110	FixedWing.Bf 110.obj	O
Blenheim	FixedWing.Mosquito.obj	N
BR20	FixedWing.Bf 110.obj	N
CR42	FixedWing.CAML.obj	N
D520	FixedWing.LaGG-3.obj	N
Defiant	FixedWing.Hurricane.obj	N
DH82	FixedWing.Dolphin.obj	N
Do17	FixedWing.B-25.obj	N
Do215	FixedWing.B-25.obj	N

FW200	FixedWing.B-17.obj	N
G50	FixedWing.I-16.obj	N
Gladiator	FixedWing.Eagle II.obj	N
He111	FixedWing.He 111.obj	O
Hurricane	FixedWing.Hurricane.obj	O
Ju87	FixedWing.Ju 87.obj	O
Ju88	FixedWing.Ju 88.obj	O
Kittyhawk	FixedWing.P-40.obj	O
C202	FixedWing.C.202.obj	O
Martlet	FixedWing.M20.obj	N
Spitfire	FixedWing.Spitfire.obj	O
Su26	FixedWing.Edge 540.obj	N
Sunderland	FixedWing.Mosquito.obj	N
Tomahawk	FixedWing.P-40.obj	O
Walrus	FixedWing.Mosquito.obj	N
Wellington	FixedWing.Mosquito.obj	N

If a graphic designer wish model the missing 3D models, it is possible to add our own custom models, but this will require adding them to a special Tacview folder.

For more information, visit <https://www.tacview.net/documentation/3dobjects/en/>

Useful links.

Tacview software.

<https://www.tacview.net/product/en/>

Tacview's Wiki.

https://tacview.fandom.com/wiki/Tacview_Wiki

Tacview's GitHub.

<https://github.com/Vyrtuoz/Tacview>

ATAG Forum.

<https://theairtacticalassaultgroup.com/forum/>

ATAG Forum, Full Mission Builder and Scripts section.

<https://theairtacticalassaultgroup.com/forum/forumdisplay.php?f=50>

Multiplayer server configuration.

https://artist.onlyoffice.eu/Products/Files/DocEditor.aspx?fileid=6541916&doc=c0dUQ0Zta01tS0MyYjVqeJRxU2hmeloyVXNLU0s2eDZlbit0TFZYblFNUT0_ljY1NDE5MTYiQ

Mission resources (scripts).

<https://theairtacticalassaultgroup.com/forum/showthread.php?t=34571>

Watchdog.

<https://theairtacticalassaultgroup.com/forum/showthread.php?t=35224&highlight=watchdog>
https://www.team-fusion-simulations.de/tfsresources/?&path=03_Servers

Viewer for obj format files (3D geometry).

<http://www.cemyuksel.com/software>

.NET library for compressing files in ZIP format.

<https://documentation.help/DotNetZip/About.htm>

Known issues.

- The ballistics of projectiles (bullets, shells, bombs, torpedoes) are not available and are not represented in Tacview.
- The recorder does not work for training and quick missions because these missions are auto-generated when they are launched.
- Some objects have no correspondence in [Tacview database of 3D objects](#). This is mainly a problem for static objects and buildings, and to a lesser extent for aircraft, ships, and vehicles (tanks, trains, etc.). For aircraft, the best possible match (shape, size) available for their representation in Tacview was chosen.
- The size of the Hurricane is not correct in Tacview version 1.9.3 and earlier. The problem has been solved in version 1.9.4 beta 4 ([requires a standard license](#)).
- Slight discrepancy in the geographical position of objects in Tacview. I'm working on it but if a cartographer is available, I'm open to collaboration.
- Orientation of static objects is incorrect. This data is not available for this type of object.
- Destruction of certain buildings does not trigger the OnBuildingKilled event.
- There may still be a few hidden bugs. Don't hesitate to report them on the [ATAG](#) forum.